

Lab: Extreme Events

Modeling the Unusual

- For a given sample, the maximum and minimum are fixed
- But if we have a large number of samples, we can study the distribution/density of those max/min draws
- Imagine:
 - The biggest storm in a year
 - The worst loss for the market in a month
 - The biggest earthquake in a decade
 - The longest time between goals in a game
- This is how we understand the distribution/density of extreme events
- The k -th order statistic is the k -th largest value in a sample of n from some distribution
- The minimum is the first order statistic and the maximum is the n -th order statistic

Example: Drawing from a Gaussian

Below, we find the minimum of a sample of size n drawn from a normal distribution, and then repeat this b times, to get a distribution of the minimum with 10000 counts in it. This is the distribution of the 1st order statistic, shown below in blue.

```
In [3]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

rng = np.random.default_rng( seed = 100 ) # Create a random number stream

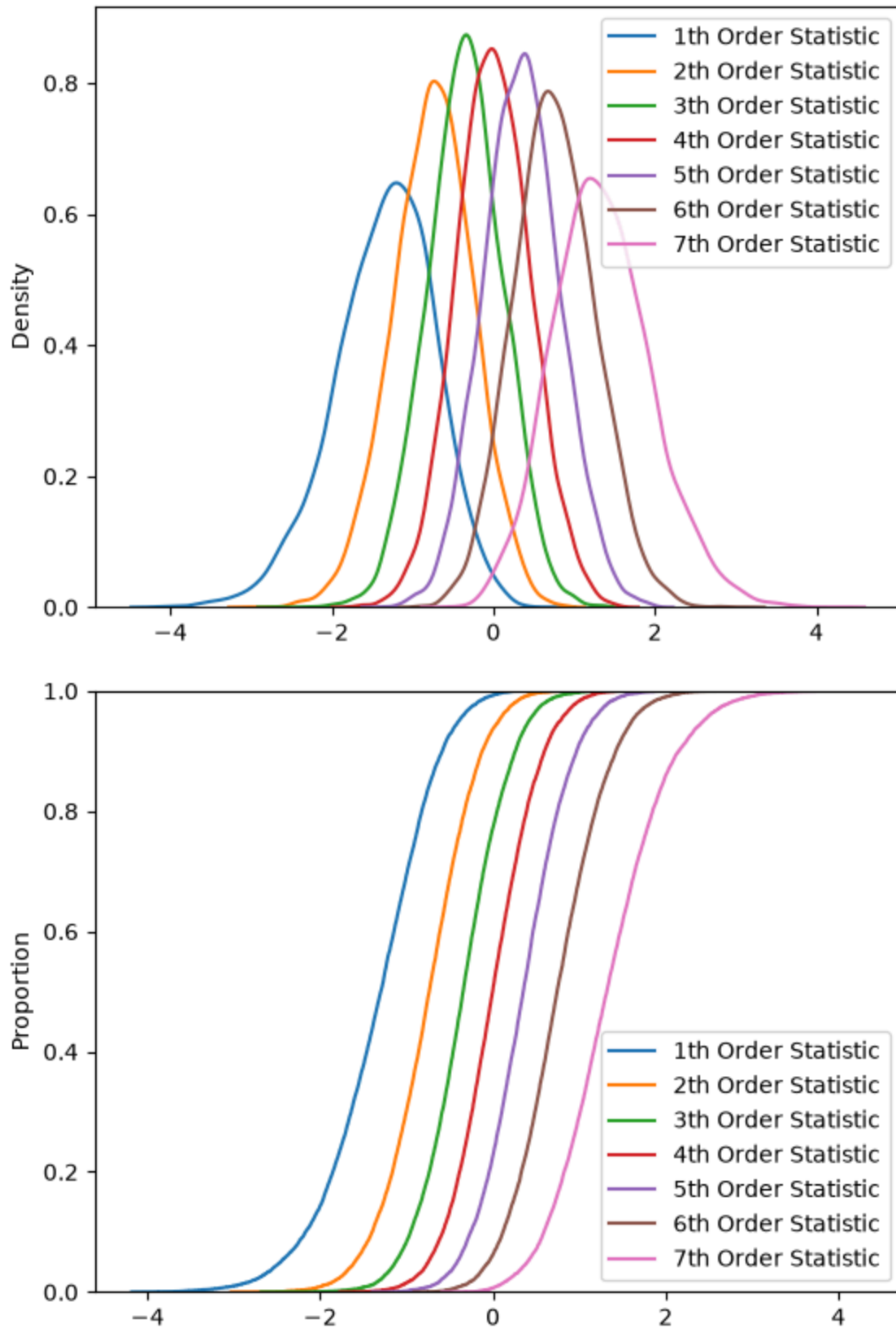
n = 7 # Sample Size
b = 10_000 # Number of Simulations

sample = rng.standard_normal( size = (n,b) ) # Draw a sample from the standard normal
sorted_sample = np.sort(sample, axis=0) # Sort each column (each simulated sample)

for j in range(n): # For each rank
    sns.kdeplot( sorted_sample[j,:], label=f'{1+j}th Order Statistic') # Plot the density
plt.legend() # Add legend
plt.show()

for j in range(n): # For each rank
    sns.ecdfplot( sorted_sample[j,:], label=f'{1+j}th Order Statistic') # Plot the empirical CDF
```

```
plt.legend() # Add legend  
plt.show()
```



Analytic Relationship

Maximum

If $F(x)$ is the underlying true CDF, let's consider the analytical equation for the distribution of the maximum of a sample of size n . (The maximum is its own random variable, with its distribution plotted above. What is this in math?) To build the CDF of the maximum, we ask (as always): "What's the probability the maximum lands at or below x ?" for every x . In other words:

$$F_{\max}(x) = P(X_{\max} \leq x)$$

This is nothing new, just applying it to the distribution of a specific order statistic, the max.

So what is the **probability that all of the draws are less than or equal to x** , i.e. what is the probability that the maximum is less than or equal to x (the right hand side)?

Because each draw is independent:

$$P(X_1 \leq x \wedge \dots \wedge X_n \leq x) = P(X_1 \leq x) \cdot P(X_2 \leq x) \cdot \dots \cdot P(X_n \leq x)$$

Because we are pulling from the same distribution every time, **each of these probabilities equals our CDF, $F(x)$** :

$$P(X_1 \leq x, \dots, X_n \leq x) = F(x) \cdot F(x) \cdot \dots \cdot F(x)$$

The probability that all of the draws are less than or equal to x is therefore:

$$F_{\max}(x) = F(x) \cdot F(x) \cdot \dots \cdot F(x) = F(x)^n$$

What is its density?

Minimum

The distribution of the minimum is somewhat harder, because our CDF is asking $\leq x$, and " $\max \leq x$ " means " $\text{all} \leq x$ " but " $\min \leq x$ " does not mean " $\text{all} \leq x$ ". We can't just substitute in multiple copies of $F(x)$. Instead of looking at the probability that all draws are less than or equal to x , we need to know the probability that at least one is.

We want to ask "is the minimum less than or equal to x ?" which is the opposite of asking "are all of the draws, including the minimum, above x ?" The probability that all observations are above x is $(1 - F(x))^n$. Because this is the opposite of the question we actually want to ask, the complementary probability is what we truly want:

$$F_{\min}(x) = 1 - (1 - F(x))^n$$

What is its density?

All of the other order statistics, generally

- If you're curious, the density of the k -th of n order statistics is

$$f_k(x) = \frac{n!}{(k-1)!(n-k)!} F(x)^{k-1} (1-F(x))^{n-k} f(x)$$

- We'll use the results for the minimum and maximum

The Data

We have three datasets:

1. Rainfall *totals* for every state-month from 1981 to 2025, with 5 reporting stations per state each month
2. Rainfall for every *station-day* over that same 1981-2025 period (same 5 stations per state) — much bigger, and used for a single question near the end of Step One
3. Daily and monthly U.S. stock market returns from 1926 to 2026

We want to understand tail behavior, to better reason about risk (engineering, financial, etc.).

```
In [4]: df = pd.read_parquet("data/ghcn_prcp_monthly.parquet")
df.head()
```

```
Out [4]:
```

	state	state_name	station_rank	station_id	station_name	latitude	longitude
0	AK	Alaska	1	USW00027502	BARROW AP	71.2869	-156.7394
1	AK	Alaska	1	USW00027502	BARROW AP	71.2869	-156.7394
2	AK	Alaska	1	USW00027502	BARROW AP	71.2869	-156.7394
3	AK	Alaska	1	USW00027502	BARROW AP	71.2869	-156.7394
4	AK	Alaska	1	USW00027502	BARROW AP	71.2869	-156.7394

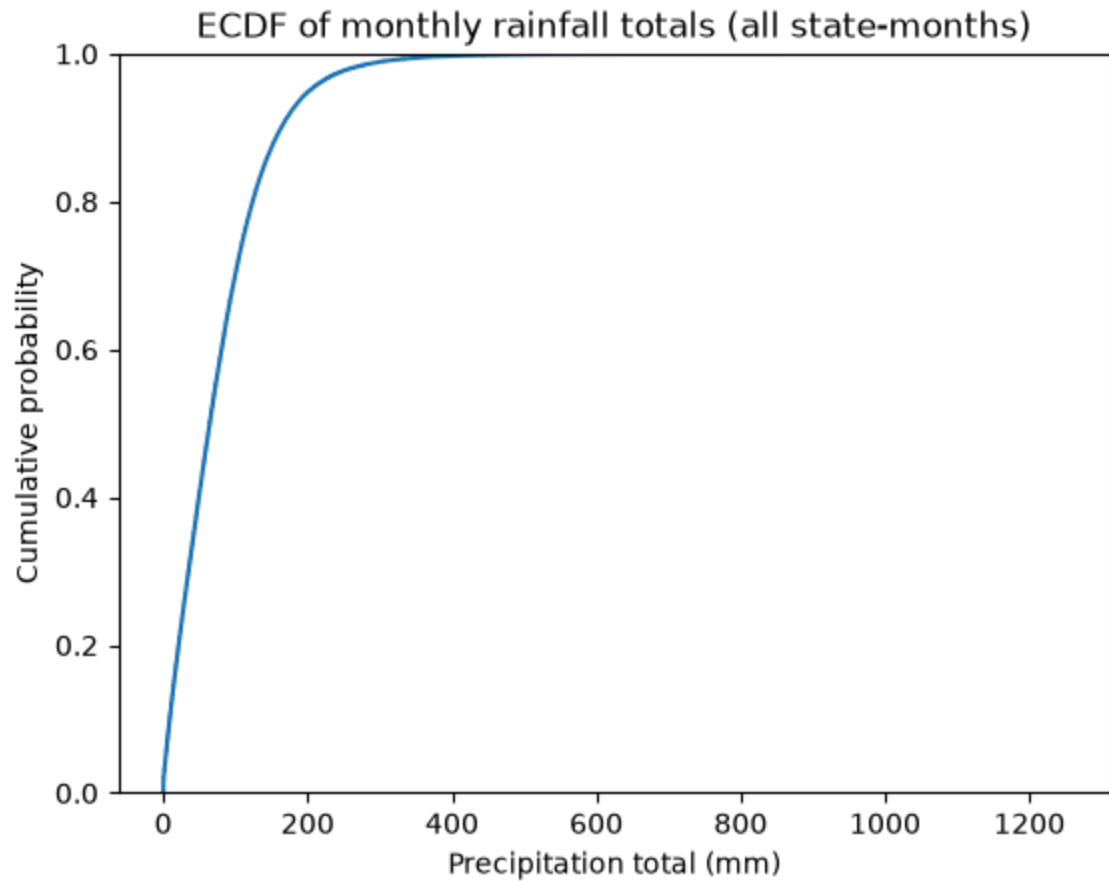
Step One.

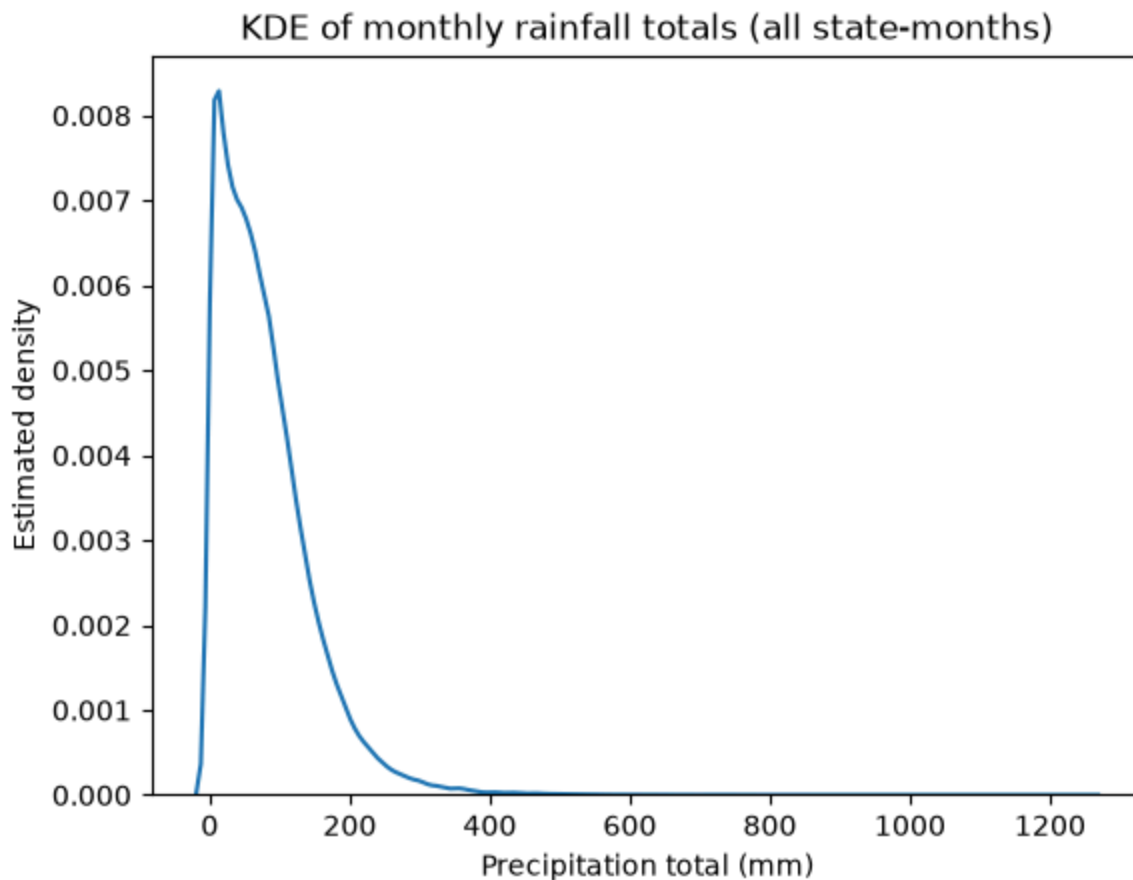
1. Plot the ECDF and KDE of `prcp_total_mm` with seaborn— the distribution and density of rainfall across the entire U.S.

```
In [5]: sns.ecdfplot(data=df, x="prcp_total_mm")
plt.xlabel("Precipitation total (mm)")
plt.ylabel("Cumulative probability")
plt.title("ECDF of monthly rainfall totals (all state-months)")
plt.show()

sns.kdeplot(data=df, x="prcp_total_mm")
plt.xlabel("Precipitation total (mm)")
plt.ylabel("Estimated density")
```

```
plt.title("KDE of monthly rainfall totals (all state-months)")  
plt.show()
```





2. Now group by year-month and compute the **median** and **maximum** precipitation nationwide. Plot the ECDF and KDE of these two statistics.

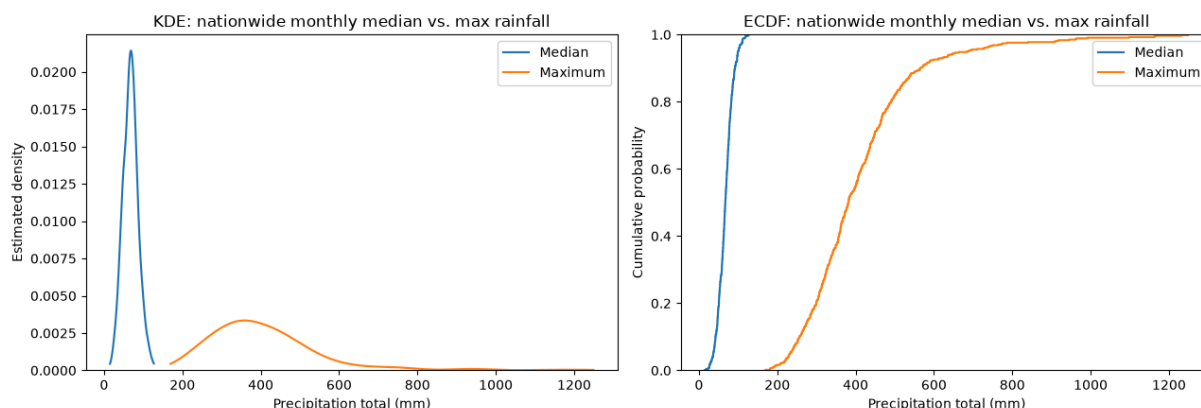
```
In [6]: monthly_nationwide = (
    df.groupby(["year", "month"], as_index=False)
      .agg(
        median_prpc_mm=("prcp_total_mm", "median"),
        max_prpc_mm=("prcp_total_mm", "max"),
      )
)

fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))

sns.kdeplot(data=monthly_nationwide, x="median_prpc_mm", cut=0, label="Median")
sns.kdeplot(data=monthly_nationwide, x="max_prpc_mm", cut=0, label="Maximum")
axes[0].set(title="KDE: nationwide monthly median vs. max rainfall",
            xlabel="Precipitation total (mm)", ylabel="Estimated density")
axes[0].legend()

sns.ecdfplot(data=monthly_nationwide, x="median_prpc_mm", label="Median", ax=axes[0])
sns.ecdfplot(data=monthly_nationwide, x="max_prpc_mm", label="Maximum", ax=axes[1])
axes[1].set(title="ECDF: nationwide monthly median vs. max rainfall",
            xlabel="Precipitation total (mm)", ylabel="Cumulative probability")
axes[1].legend()
```

```
plt.tight_layout()
plt.show()
```



3. How do your results in part 2 compare to your results in part 1?

Compare and contrast the ECDFs and KDEs from parts 1 and 2. Explain why they are different:

Answer:

The plots in part 1 describe a single state-month. Most state-months get a small to moderate amount of rain, with a few very wet ones, so the KDE is a tall peak at low values with a long right tail, and the ECDF reaches about 0.95 by roughly 200 mm.

The plots in part 2 describe two different quantities, each computed from a few hundred state-months in a given calendar month. The median is the middle value, so it stays steady from month to month. Its KDE is a narrow peak around 70 mm and its ECDF rises steeply. The maximum is the largest value, so it is almost always far above a typical state-month. Its KDE is centered around 350–400 mm with a wide, right-skewed spread, and its ECDF is shifted far to the right.

They differ because they answer different questions: how much rain a single state-month gets, how much the typical state-month gets, and how much the wettest state-month gets. The more draws we take, the closer the median stays to the middle and the larger the maximum becomes.

4. Now define a state's rainfall differently: take the **median** of the 5 stations in each month, then find the **maximum** across the state's 12 months in a year — the "wettest month." For each state, what's the expected value of the wettest month? Plot the ECDF of Virginia's annual maxima (the wettest month in each of the 45 years), and overlay the analytical prediction $F_{\max}(x) = \hat{F}(x)^{12}$.

```

In [7]: # One rainfall value per state-month, taking the median of the stations
state_month = (
    df.groupby(["state", "year", "month"], as_index=False)
        .agg(prcp_total_mm=("prcp_total_mm", "median"))
) # TODO median calculation

# Maximum of the 12 state-month observations above, for each year: the wettest
state_year_max = (
    state_month.groupby(["state", "year"], as_index=False)
        .agg(prcp_total_mm=("prcp_total_mm", "max"))
) # TODO max calculation

exp_wettest = state_year_max.groupby("state", as_index=False).agg(E_wettest=
exp_wettest.loc[exp_wettest["state"] == "VA"]

state = "VA"

levels = (
    state_month.loc[state_month["state"] == state, "prcp_total_mm"]
        .to_numpy()
)

observed_max = (
    state_year_max.loc[state_year_max["state"] == state, "prcp_total_mm"]
        .to_numpy()
)

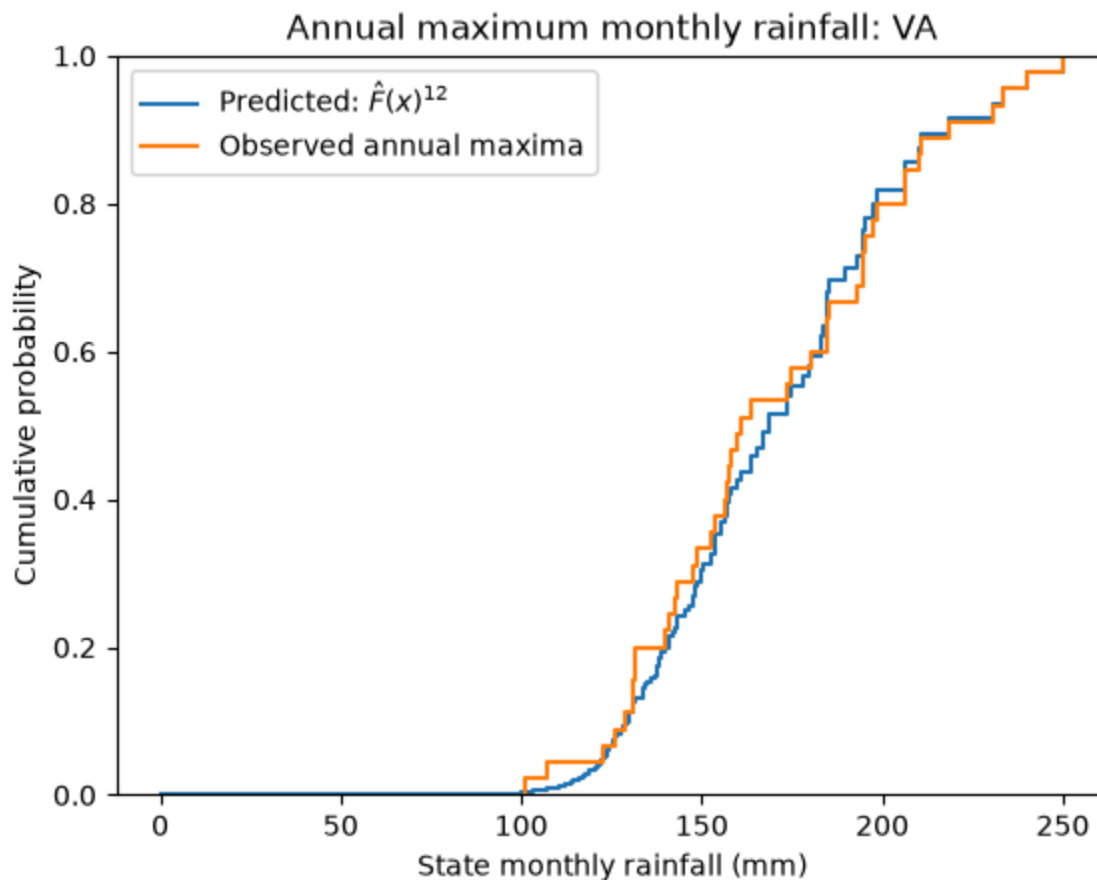
x = np.sort(np.unique(levels))
F_hat = np.searchsorted(np.sort(levels), x, side="right") / len(levels)

plt.step(
    x,
    F_hat**12,
    where="post",
    label=r"Predicted: $\hat{F}(x)^{12}$"
)

sns.ecdfplot(x=observed_max,
    label="Observed annual maxima")

plt.xlabel("State monthly rainfall (mm)")
plt.ylabel("Cumulative probability")
plt.title(f"Annual maximum monthly rainfall: {state}")
plt.legend()
plt.show()

```



How close is the fit here? If it differs from question 4's fit, why:

Answer. The predicted curve $\hat{F}(x)^{12}$ matches the observed ECDF of Virginia's annual maxima very closely. The largest vertical gap is about 0.075, and the observed mean wettest month (about 170 mm) is nearly the same as the predicted mean (about 172 mm).

5. **A question for FEMA.** So far you've worked with *monthly* rainfall totals. Switch now to the daily station-level file (`ghcn_prcp_daily.parquet`) — reload `df` from it, and don't mix it with the monthly data above.

A day with more than 120mm of rain is a serious flood/landslide risk. On any single station-day, that's rare. But a state doesn't get one chance a month — it gets 5 stations $\times$ (days in the month) chances. FEMA doesn't care about the average day, or the single worst day on record; it cares about the probability that *some* station has a disaster day at some point during the month.

For each state-month, estimate

$$P(\text{at least one station-day exceeds 120mm}) = 1 - \hat{F}(120)^n$$

where $\hat{F}$ is the empirical CDF of a single station-day's rainfall for that state and calendar month (pooled across all years), and n is the number of station-days in one occurrence of that month (5 stations $\times$ days in the month). Report the 20 highest-risk state-months.

```
In [13]: # Fresh load -- this question uses only the daily file, not the monthly one
#df_daily = pd.read_parquet("data/ghcn_prdp_daily.parquet")

url = (
    "https://www.dropbox.com/scl/fi/"
    "pjmyop3fqjdy8oabkc4ab/ghcn_prdp_daily.parquet"
    "?rlkey=otxenczb3pxgzz53kc1i5zxww&dl=1"
)
df_daily = pd.read_parquet(url)
df_daily = df_daily.loc[df_daily["observed"]]

threshold = 120 # mm in a single day

def disaster_risk(group):
    F_hat = (group["prdp_mm"] <= threshold).mean()
    n = len(group) / group["year"].nunique() # station-days in one year's c
    return 1 - F_hat**n

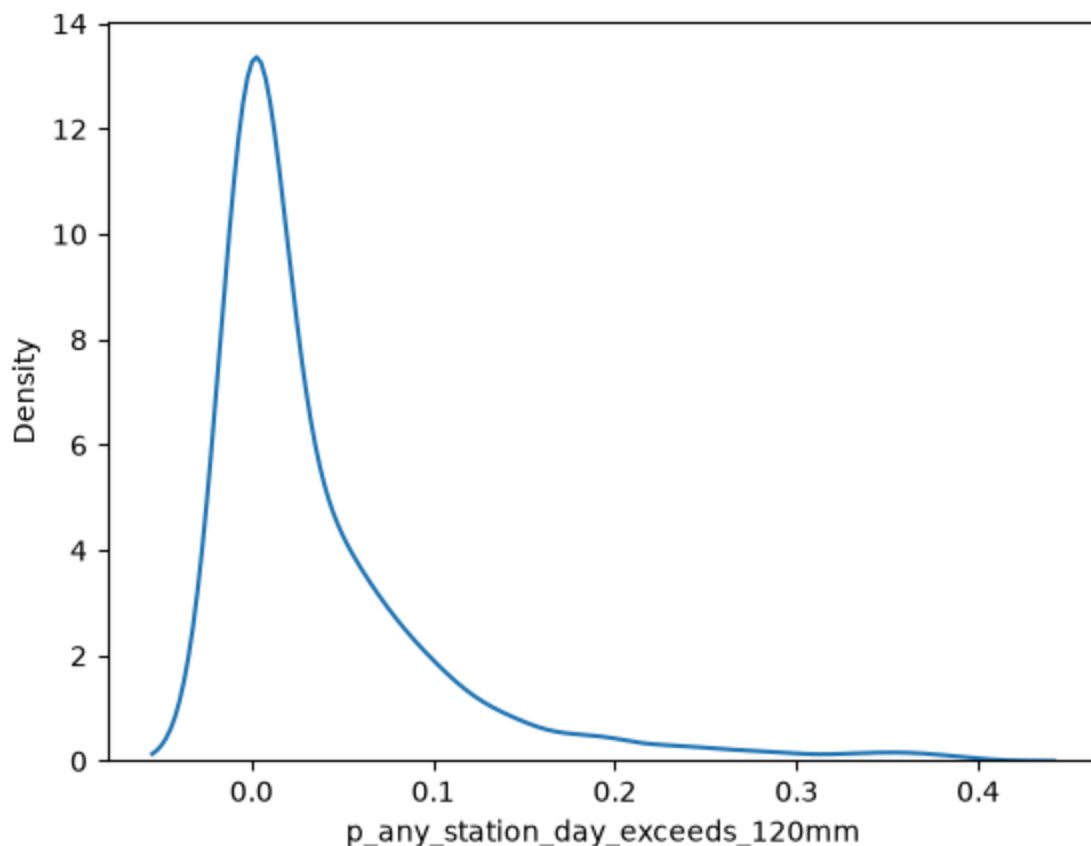
risk = (
    df_daily.groupby(["state", "month"])
    .apply(disaster_risk, include_groups=False)
    .rename("p_any_station_day_exceeds_120mm")
    .reset_index()
    .sort_values("p_any_station_day_exceeds_120mm", ascending=False)
)

sns.kdeplot(risk['p_any_station_day_exceeds_120mm'])

risk.head(20)
```

Out[13]:

	state	month	p_any_station_day_exceeds_120mm
122	HI	3	0.387166
104	FL	9	0.373367
130	HI	11	0.359242
208	LA	5	0.359229
131	HI	12	0.344789
121	HI	2	0.330060
105	FL	10	0.330026
476	SC	9	0.299512
207	LA	4	0.299512
319	NC	8	0.283726
536	VA	9	0.267604
101	FL	6	0.267604
211	LA	8	0.267597
209	LA	6	0.251113
212	LA	9	0.234253
127	HI	8	0.234248
215	LA	12	0.234248
117	GA	10	0.234248
103	FL	8	0.234248
120	HI	1	0.217012



Which states and months face the most risk of a greater-than-120mm day? How high is the probability? Among all state-months, describe the density of high precipitation days:

Answer. The highest-risk state-months are in **Hawaii, Louisiana and Florida**. Hawaii has six of the top 20 (March is the highest at about 39%, followed by November, December, February, August and January), Louisiana has six (led by May at about 36%, with April, June, August, September and December), and Florida has four (September at about 37%, plus October, June and August). South Carolina (September), North Carolina (August), Georgia (October) and Virginia (September) appear once each. The list mixes Atlantic and Gulf hurricane months (June–October) with Hawaii's winter and spring storm months.

The probabilities are high. The top state-month, Hawaii in March, has about a 39% chance that at least one station records a day above 120 mm, and even the 20th highest (Hawaii in January) is about 22%. Only a tiny fraction of individual station-days exceed 120 mm, but a state has about 150 chances in a month (5 stations × about 30 days), so $1 - \hat{F}(120)^n$ becomes large even when each single-day chance is very small.

Across all state-months, the density of this risk is strongly right-skewed. Most state-months have a probability at or very near zero (dry states and dry seasons almost

never see a 120 mm day), and a small number of state-months form a thin tail out to the 20–40% values listed above. High risk is therefore concentrated in a few places and seasons rather than spread evenly across the country.

Step Two.

Our second data set is daily and monthly U.S. stock market returns. Instead of storms across space (five stations in a state) or across the calendar (twelve months in a year), the "sample" is the set of trading days within a month, and we ask about the best and worst day in that sample.

```
In [9]: df = pd.read_parquet("data/ff_market_daily.parquet")

print(f"{len(df):,} trading days")
print(f"{df['date'].min().date()} through {df['date'].max().date()}")
df.head()
```

26,296 trading days
1926-07-01 through 2026-07-31

```
Out[9]:
```

	date	year	month	month_start	trading_day_of_month	mkt_excess_return_pct
0	1926-07-01	1926	7	1926-07-01	1	0.09
1	1926-07-02	1926	7	1926-07-01	2	0.45
2	1926-07-06	1926	7	1926-07-01	3	0.17
3	1926-07-07	1926	7	1926-07-01	4	0.09
4	1926-07-08	1926	7	1926-07-01	5	0.22

1. Plot the KDE and ECDF of daily market returns.

```
In [10.. fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))

sns.kdeplot(
    data=df,
    x="market_return_pct",
    cut=0,
    ax=axes[0],
)
axes[0].axvline(0, color="black", linewidth=1, alpha=0.6)
axes[0].set(
    title="KDE of daily U.S. market returns",
```

```

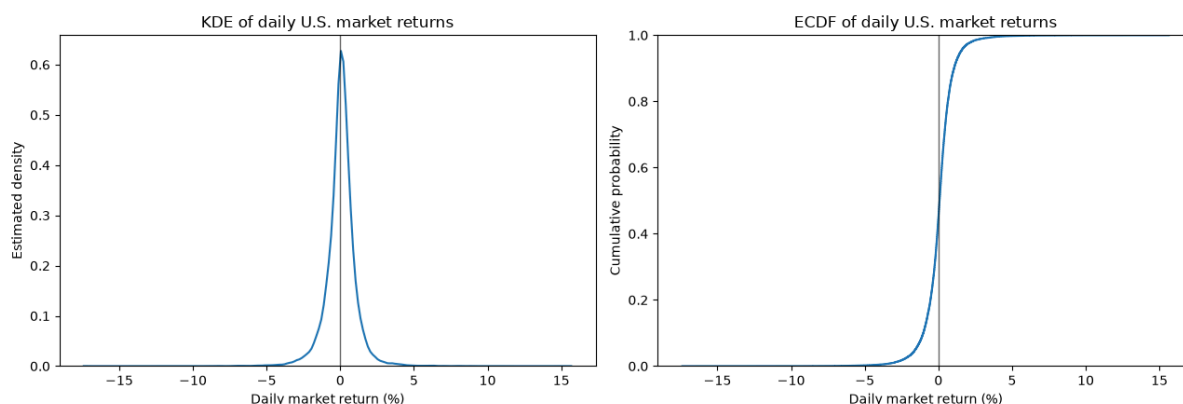
    xlabel="Daily market return (%)",
    ylabel="Estimated density",
)

sns.ecdfplot(
    data=df,
    x="market_return_pct",
    ax=axes[1],
)

axes[1].axvline(0, color="black", linewidth=1, alpha=0.6)
axes[1].set(
    title="ECDF of daily U.S. market returns",
    xlabel="Daily market return (%)",
    ylabel="Cumulative probability",
)

plt.tight_layout()
plt.show()

```



- For each calendar month, compute the **best** (max) and **worst** (min) daily return — the first and n -th order statistics of that month's trading days. Plot the KDE of the monthly best and worst returns, overlaid.

```

In [11... monthly = (
    df.groupby(["year", "month"], as_index=False)
        .agg(
            month_start=("month_start", "first"),
            trading_days=("date", "size"),
            worst_return_pct=("market_return_pct", "min"),
            best_return_pct=("market_return_pct", "max"),
            median_return_pct=("market_return_pct", "median"),
        )
)

# Put losses and gains on the same positive-magnitude scale.
monthly["worst_loss_pct"] = -monthly["worst_return_pct"]

print(f"{len(monthly):,} calendar months")
print(
    f"Trading days per month: {monthly['trading_days'].min()} "
    f"to {monthly['trading_days'].max()}"
)

```

```
)
monthly.head()
```

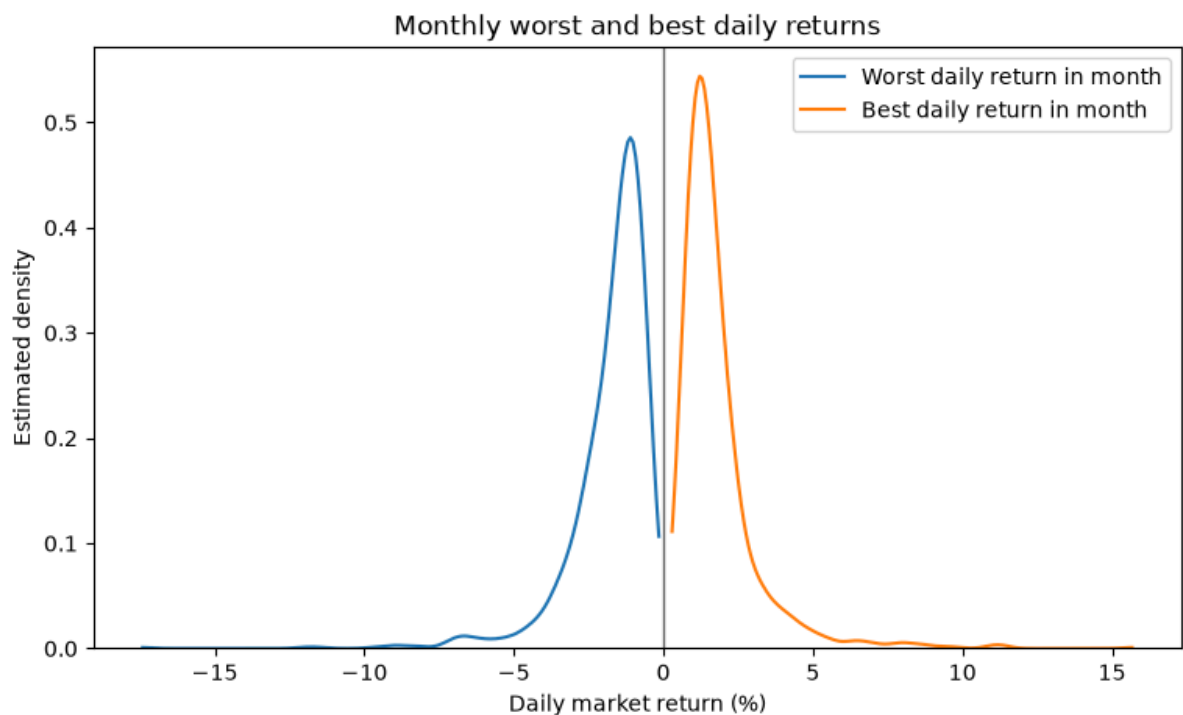
1,201 calendar months

Trading days per month: 15 to 27

Out[11]:

	year	month	month_start	trading_days	worst_return_pct	best_return_pct	mea
0	1926	7	1926-07-01	25	-0.72	1.10	
1	1926	8	1926-08-01	26	-1.39	0.85	
2	1926	9	1926-09-01	24	-0.98	0.91	
3	1926	10	1926-10-01	25	-1.81	1.53	
4	1926	11	1926-11-01	24	-0.62	0.67	

```
In [12... plt.figure(figsize=(9, 5))
sns.kdeplot(
    x=monthly["worst_return_pct"],
    label="Worst daily return in month",
    cut=0,
)
sns.kdeplot(
    x=monthly["best_return_pct"],
    label="Best daily return in month",
    cut=0,
)
plt.axvline(0, color="black", linewidth=1, alpha=0.6)
plt.xlabel("Daily market return (%)")
plt.ylabel("Estimated density")
plt.title("Monthly worst and best daily returns")
plt.legend()
plt.show()
```



Describe the symmetry (or lack of it) between the best- and worst-day densities. Compare them to the overall populations of daily returns. Use code as appropriate to make these comparisons, and then write out your answer in a markdown cell.

Answer. The best-day and worst-day densities are close to each other. The worst-day KDE peaks at about -1.1% and the best-day KDE at about $+1.2\%$. The medians are nearly the same size (-1.41% vs $+1.44\%$), and so are the spreads (standard deviation about 1.4 for both) and the skewness (-3.4 vs $+3.6$). Each has a long tail pointing away from zero: the worst day extends down to -17.4% (October 1987) and the best day up to $+15.7\%$ (March 1933). The only visible asymmetry is that the single most extreme event is a crash.

Both differ sharply from the overall population of daily returns. Daily returns form a tall, narrow, nearly symmetric peak centered at about zero (mean 0.04% , median 0.08% , skew -0.17 , standard deviation about 1.08%) with heavy tails (kurtosis about 16). The monthly best and worst days are not centered near zero at all. A typical monthly worst day (-1.41%) falls at about the 6th percentile of daily returns and a typical best day ($+1.44\%$) at about the 94th, so each month's extreme lands in the tail of the daily distribution. They are also much more skewed (about ± 3.5 vs -0.17 for daily returns), because each is the smallest or largest of roughly 22 trading days, which is what the order-statistic formulas predict.

3. Losses and gains are on opposite sides of zero, which makes them hard to compare directly. Put them on the same scale by looking at the *magnitude* of the worst loss (`-worst_return_pct`) against the magnitude of the best gain (`best_return_pct`). Plot both the KDE and ECDF.

```
In [15... fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))

sns.kdeplot(
    x=monthly["worst_loss_pct"],
    label="Worst loss magnitude",
    cut=0,
    ax=axes[0],
)

sns.kdeplot(
    x=monthly["best_return_pct"],
    label="Best gain",
    cut=0,
    ax=axes[0],
)

axes[0].set(
    title="KDE: magnitudes of monthly extremes",
    xlabel="Return magnitude (%)",
    ylabel="Estimated density",
)
```

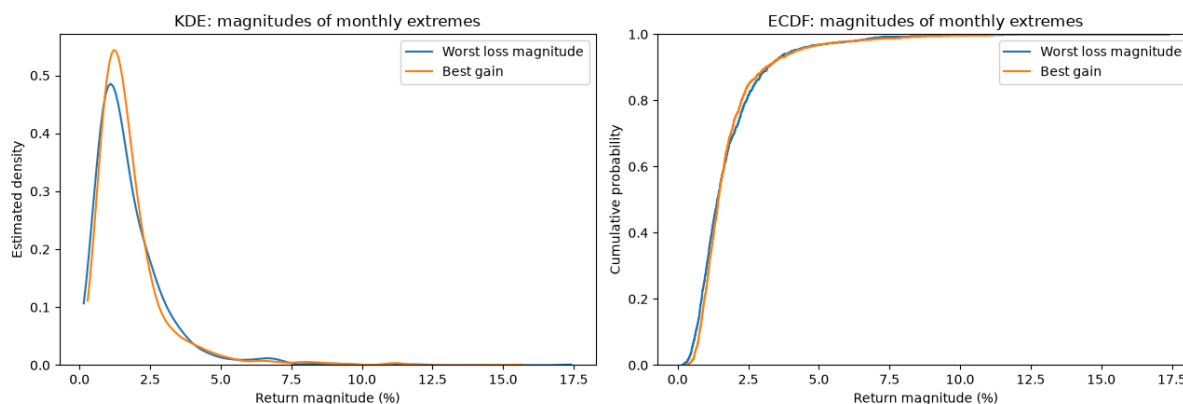
```

axes[0].legend()

sns.ecdfplot(
    x=monthly["worst_loss_pct"],
    label="Worst loss magnitude",
    ax=axes[1],
)
sns.ecdfplot(
    x=monthly["best_return_pct"],
    label="Best gain",
    ax=axes[1],
)
axes[1].set(
    title="ECDF: magnitudes of monthly extremes",
    xlabel="Return magnitude (%)",
    ylabel="Cumulative probability",
)
axes[1].legend()

plt.tight_layout()
plt.show()

```



Are large losses more extreme than large gains, or about the same:

Answer. Large losses are about as extreme as large gains. The KDEs of the worst-loss magnitude and the best-gain magnitude nearly overlap, with both peaking at about 1–1.5% and trailing off to the right, and the two ECDFs are almost indistinguishable across the whole range. The best gain is slightly more concentrated near its peak, and the worst loss has a little more mass between about 2% and 4%.

The extremes themselves differ only slightly: the single largest loss (about 17.4%, October 1987) is bigger than the single largest gain (about 15.7%, March 1933), but at the 99th percentile of monthly extremes the best gain is actually larger (about 7.9% vs 7.0%). Overall there is no consistent difference in the size of the largest monthly losses and gains.

4. Compute the median, 75th, 90th, 95th, and 99th percentile of both magnitudes from question 3. In what fraction of months does the worst loss exceed the best gain in magnitude?

```
In [16... tail_summary = pd.DataFrame(
    {
        "Worst loss magnitude": monthly["worst_loss_pct"],
        "Best gain": monthly["best_return_pct"],
    }
).quantile([0.50, 0.75, 0.90, 0.95, 0.99]).T

tail_summary.columns = ["median", "75%", "90%", "95%", "99%"]

print(
    "Fraction of months in which the worst loss exceeded the best gain: "
    f"{(monthly['worst_loss_pct'] > monthly['best_return_pct']).mean():.1%}"
)
tail_summary.round(2)
```

Fraction of months in which the worst loss exceeded the best gain: 44.8%

```
Out[16]:
```

	median	75%	90%	95%	99%
Worst loss magnitude	1.41	2.21	3.18	4.06	6.97
Best gain	1.44	2.04	3.12	4.21	7.93

What does this tail comparison say about the risk of losses vs. gains:

Answer:

The tail comparison shows that large losses and large gains are generally about the same size. The worst loss is slightly more extreme at the single maximum, but there is no consistent pattern showing that losses are always larger than gains. Therefore, the risk of extreme losses is roughly similar to the potential for extreme gains. Overall, the data does not suggest that losses are substantially more extreme than gains.

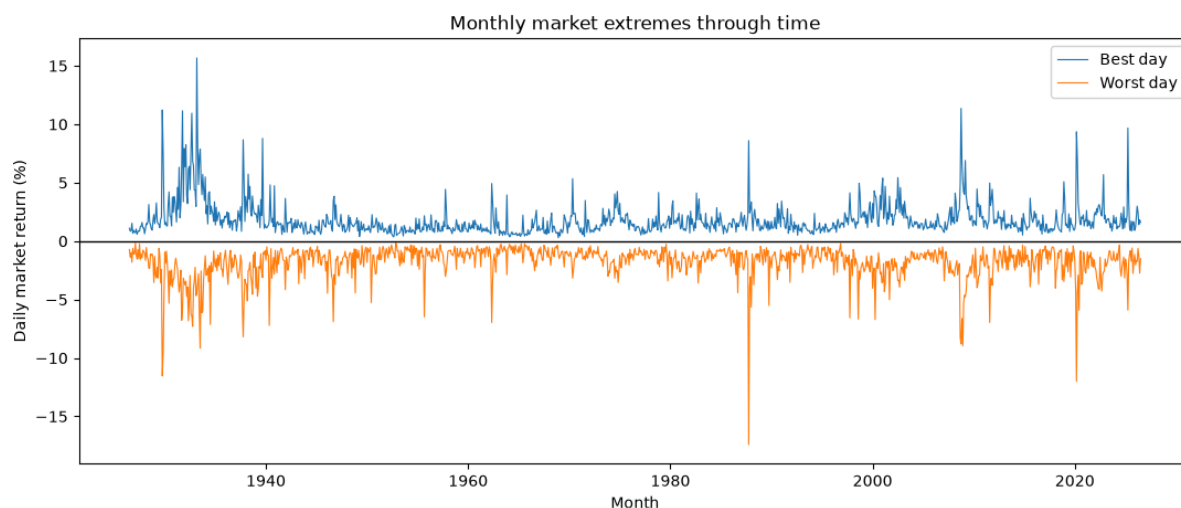
5. Plot the monthly best and worst daily returns over time. Do the extremes look stable across this ~100-year history, or are there periods where they're much larger? Can you identify any famous market events?

```
In [18... plt.figure(figsize=(13, 5))
plt.plot(
    monthly["month_start"],
    monthly["best_return_pct"],
    linewidth=0.8,
    label="Best day",
)
plt.plot(
```

```

monthly["month_start"],
monthly["worst_return_pct"],
linewidth=0.8,
label="Worst day",
)
plt.axhline(0, color="black", linewidth=1)
plt.xlabel("Month")
plt.ylabel("Daily market return (%)")
plt.title("Monthly market extremes through time")
plt.legend()
plt.show()

```



Do upside and downside risk tend to go together (volatility), or does the market exhibit distinct periods in which the good shocks are concentrated and the bad shocks are concentrated? What implication does this have for investing?:

Answer :

Upside and downside risk go together. Big gains and big losses show up in the same periods (1929-33, 1987, 2008, 2020) and both shrink in calm periods, so the market has high- and low-volatility regimes rather than separate good-shock and bad-shock periods. For investors, this means that higher potential returns can come with greater uncertainty and larger possible losses. Therefore, investors should consider volatility and diversification when managing risk.

Wrap-up

This week has focused on kernel density estimation, particularly estimating the PDF from data. This lab takes that a step further: We can estimate and quantify the densities and distributions of random variables that build off of observed phenomena. What is the difference between the ECDF and KDE? Explain how these estimations today give us different information than an ECDF or KDE plot of the

underlying data, but quantify information about the behavior of a different random variable.

Answer:

The ECDF shows the proportion of observations that are less than or equal to each value, while KDE estimates a smooth probability density from the data. An ECDF gives cumulative probabilities, whereas KDE shows the shape and density of the distribution. An ECDF or KDE of the underlying data describes a typical single observation, like one day's return. In this lab, we built new random variables from that data, like the worst loss and best gain in each month, and estimated their distributions. These are different random variables, so their ECDFs and KDEs show different things. Their typical values sit far out in the tails of the daily returns, and their distributions are more skewed. They tell us how extreme the outcomes get, which quantifies risk, and the plots of individual days can't show that.